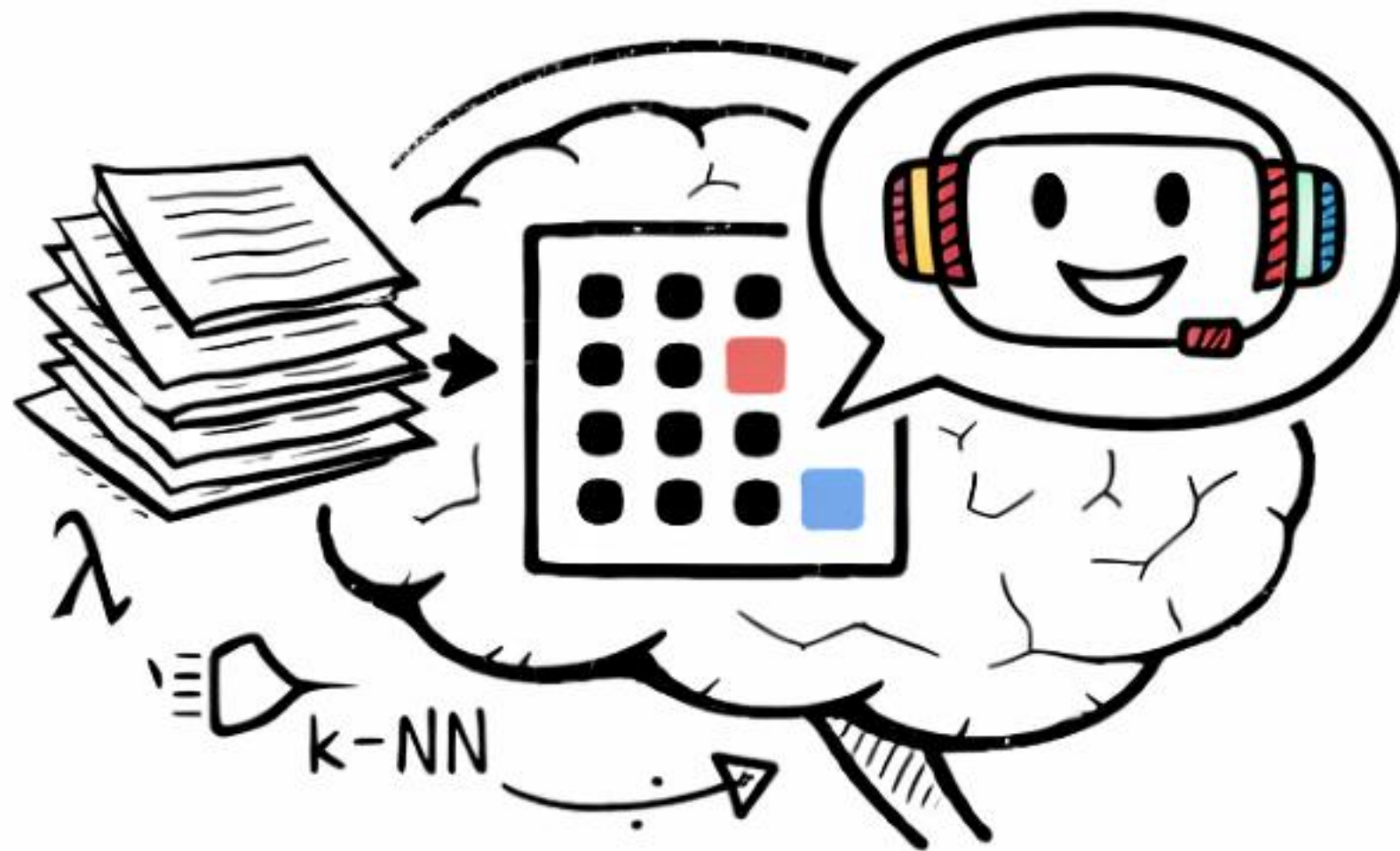


System Design:

Building a Retrieval-Augmented Knowledge Assistant



Swipe to continue →

RAG Knowledge Assistant – System Requirements

Functional Goals

Ingest & Index
~500MB
Heterogeneous Docs

Accurate, Grounded
Question Answering

Minimize Hallucinations
via Retrieval

System & Scaling Goals

Scale to Tens of
Millions of Tokens
Scalable!

Low Latency
(Interactive Use)

Semantic Chunking
(Not Fixed-Length)

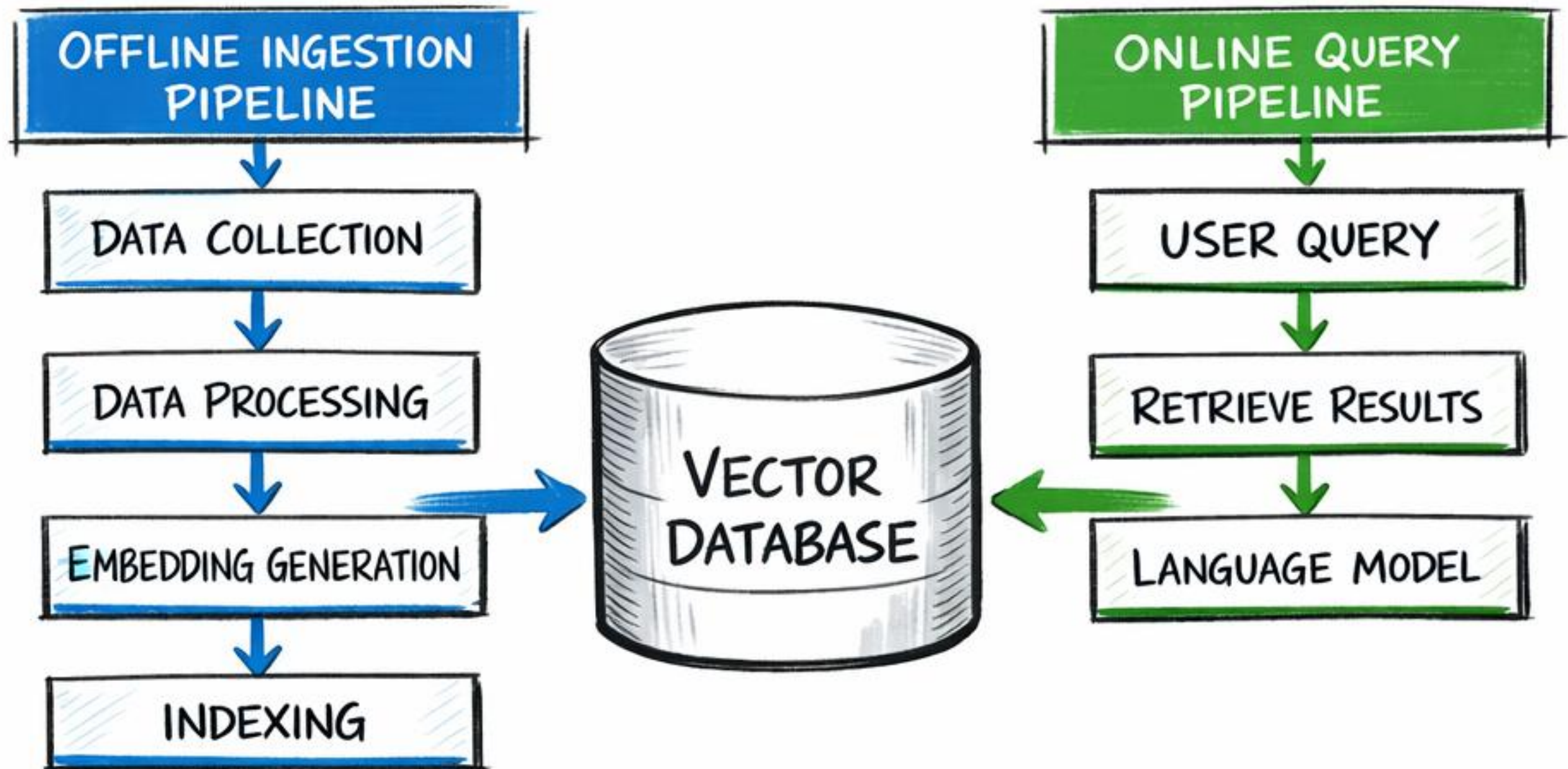
Rapid-Prototyped
Backend (e.g., FastAPI)

Similarity Search
(ANN Index)

This system defines the design constraints for a RAG assistant. It must ingest and index a large internal corpus, support scalable retrieval over millions of tokens, maintain low latency, and reduce hallucinations through grounding. Semantic chunking and deployability ensure both accuracy and real-world usability.

SYSTEM OVERVIEW:

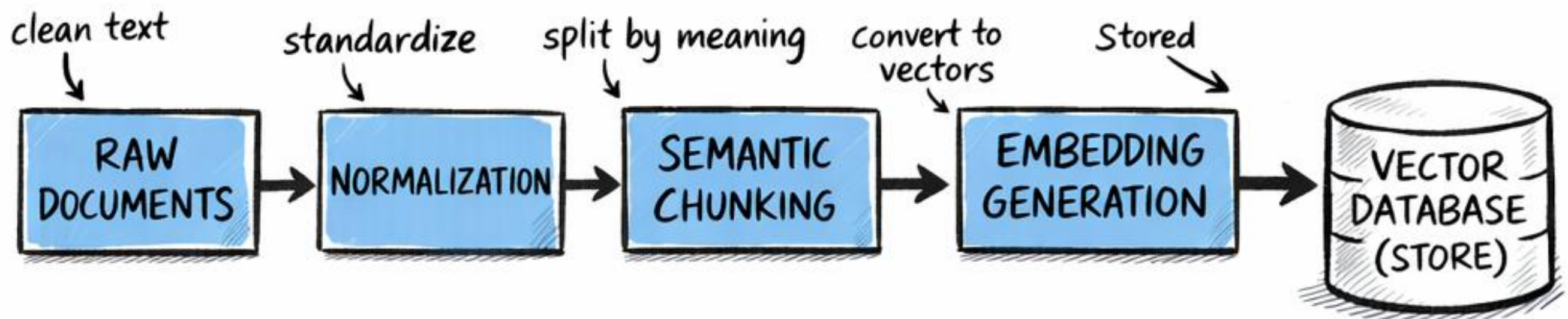
TWO-PIPELINE ARCHITECTURE



Two independent pipelines power the system. The offline ingestion pipeline prepares and stores knowledge in a vector database. The online query pipeline retrieves relevant information from that database and feeds it to a language model.

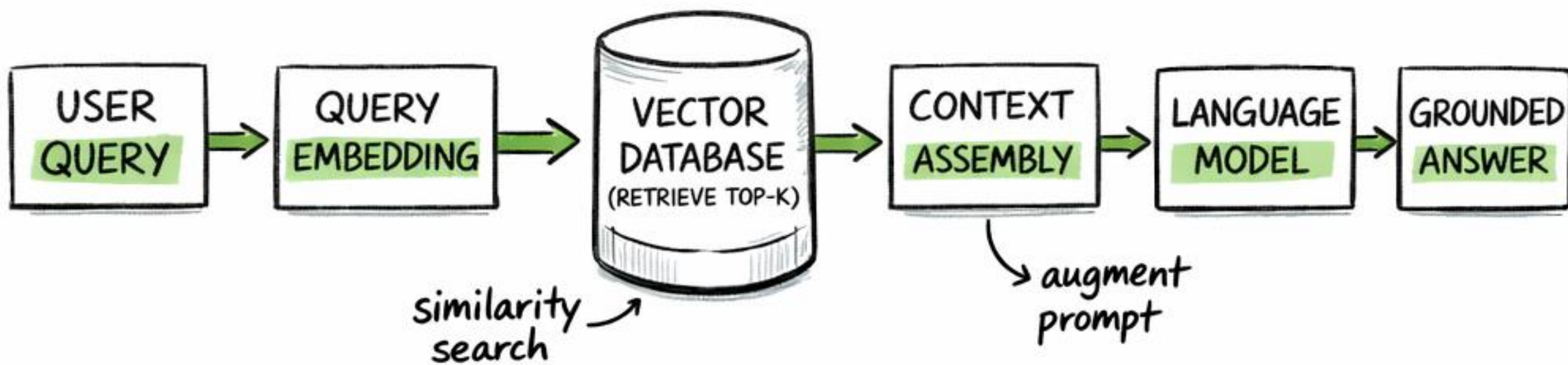
This separation enables scalability, reliability, and efficient information retrieval without mixing workloads.

OFFLINE INGESTION PIPELINE



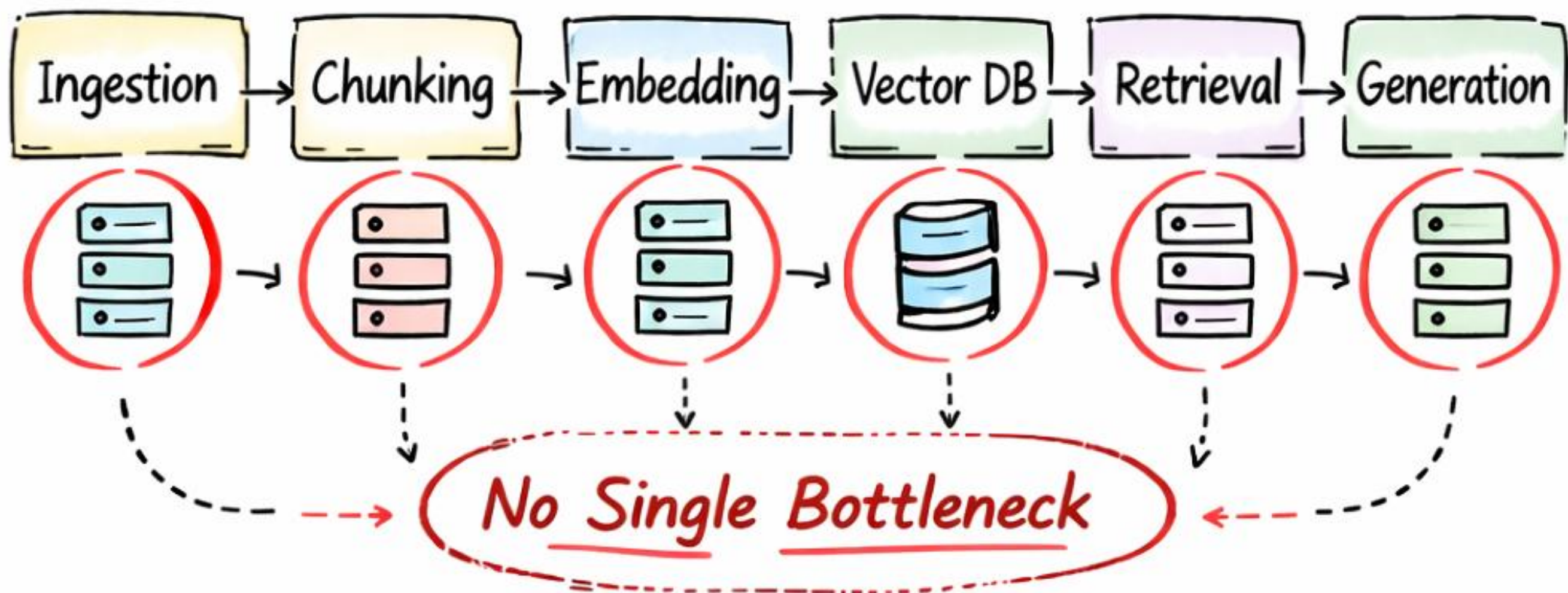
The ingestion pipeline transforms unstructured documents into searchable vector representations. Documents are cleaned and normalized, divided into semantically meaningful chunks, converted into embeddings, and stored in a vector database. This offline process prepares knowledge ahead of time, enabling fast retrieval during user queries.

ONLINE QUERY PIPELINE



The query pipeline converts a user question into an embedding, retrieves the most relevant document chunks from the vector database, assembles them into context, and supplies them to a language model. The model then generates a grounded answer based on retrieved knowledge rather than relying solely on internal memory.

INDEPENDENT SCALABILITY

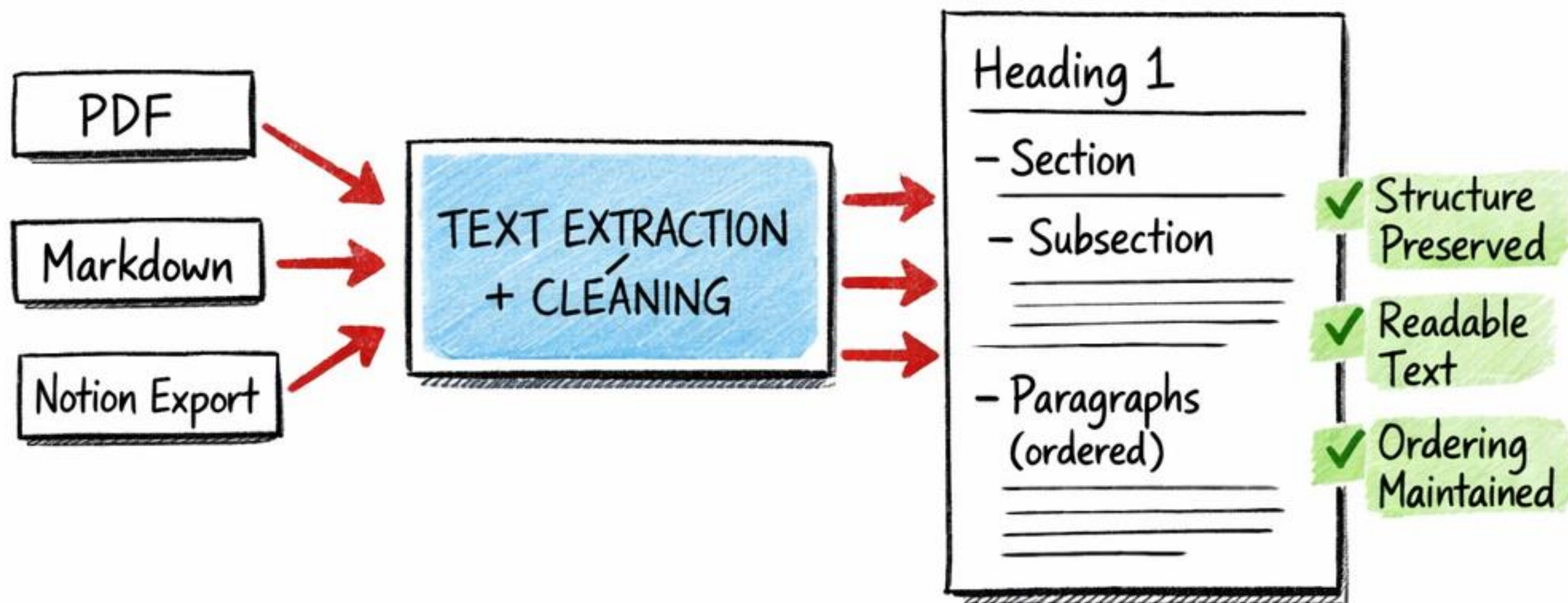


Each component in the architecture scales independently.

As document volume or query traffic grows, specific stages—such as embedding generation or retrieval—can be expanded without redesigning the entire system.

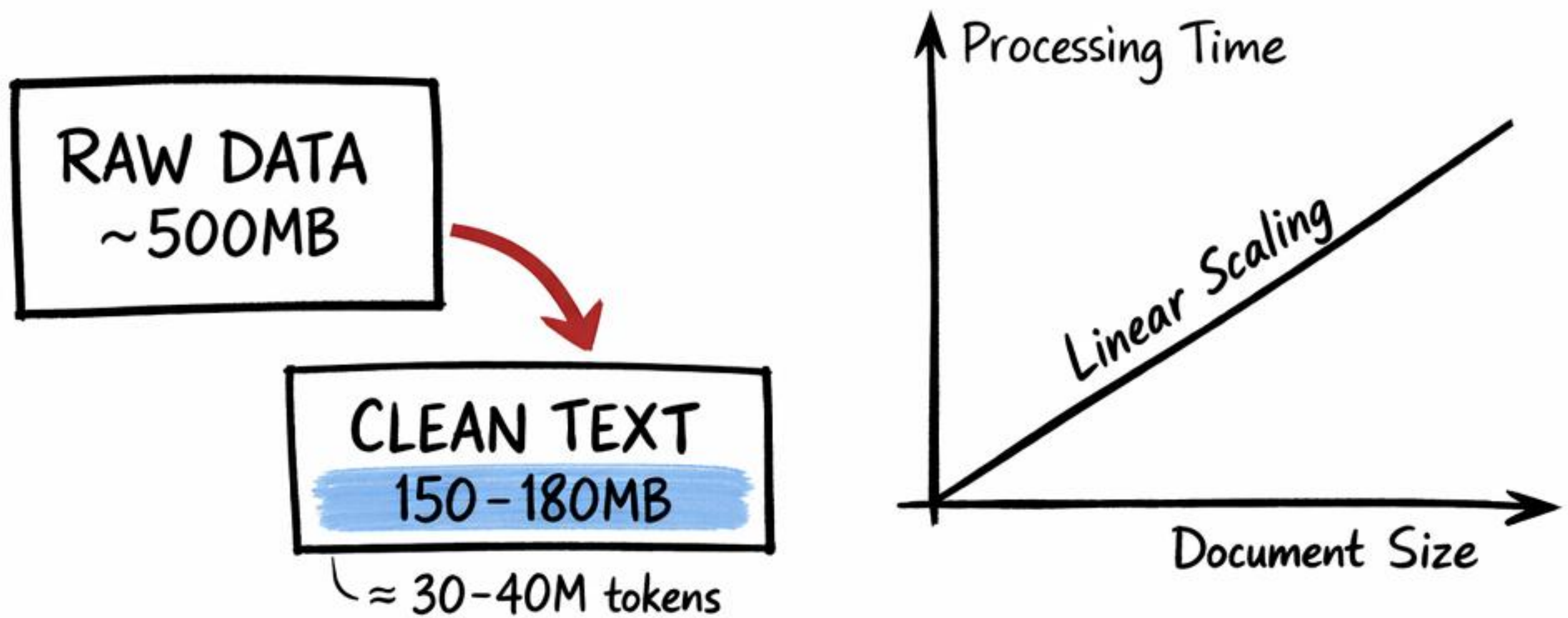
This modular design prevents bottlenecks and ensures reliable performance under increasing data and user demand.

DOCUMENT INGESTION & NORMALIZATION PIPELINE



This stage converts heterogeneous document formats into clean, structured text. While file types differ, the output is normalized into a consistent representation that preserves hierarchy and ordering. The goal is not just raw text extraction, but maintaining semantic structure so downstream systems can reason over sections and context accurately.

SCALING CHARACTERISTICS OF INGESTION



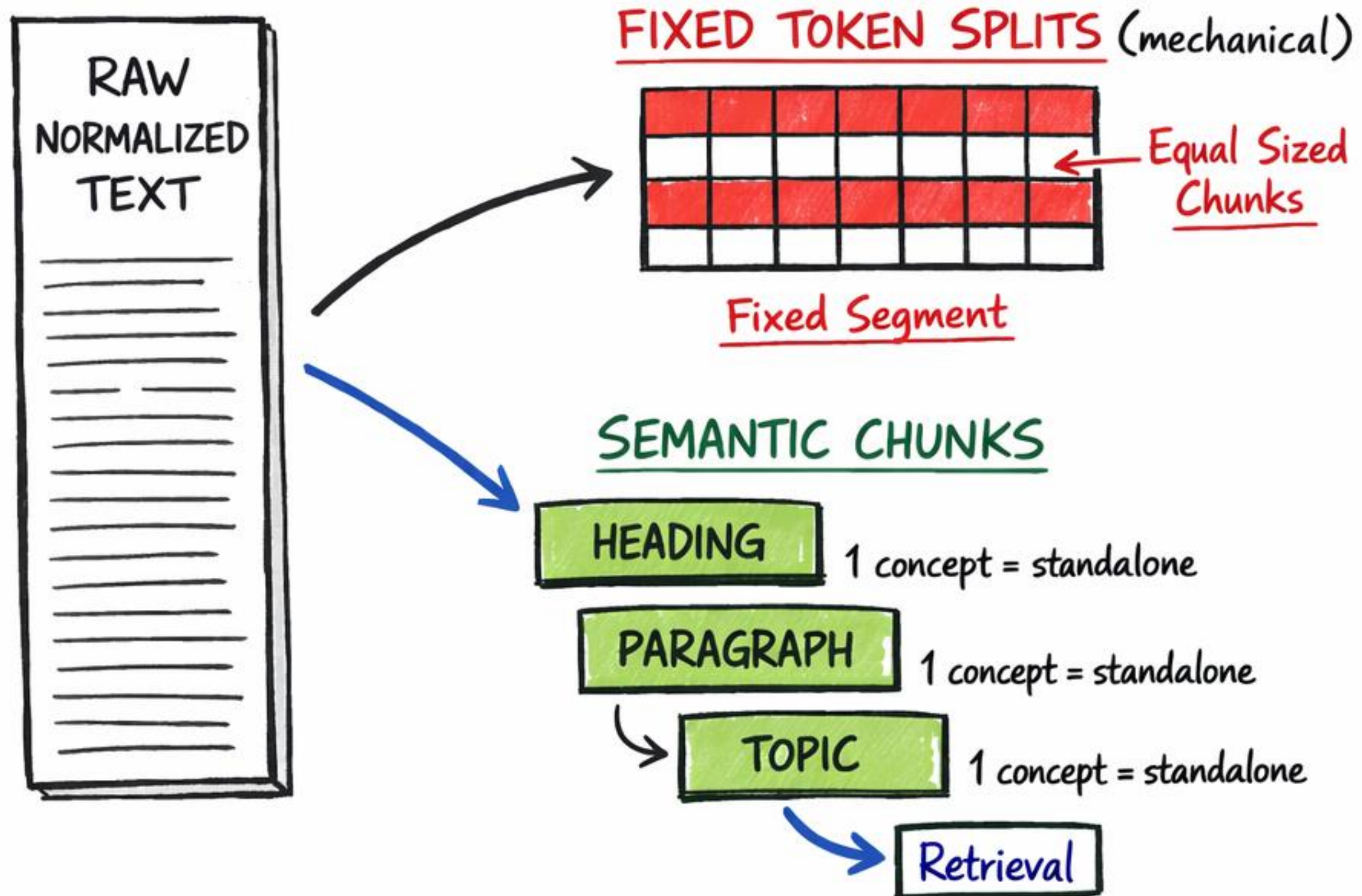
Offline Batch Job Incremental Updates

Not latency-sensitive → Safe to parallelize

Ingestion reduces storage size by removing formatting artifacts while expanding semantic clarity into tokens. Processing cost grows linearly with document size, making performance predictable. Because this step runs offline as batch or incremental jobs, it can be parallelized aggressively without impacting user-facing latency.

SEMANTIC CHUNKING

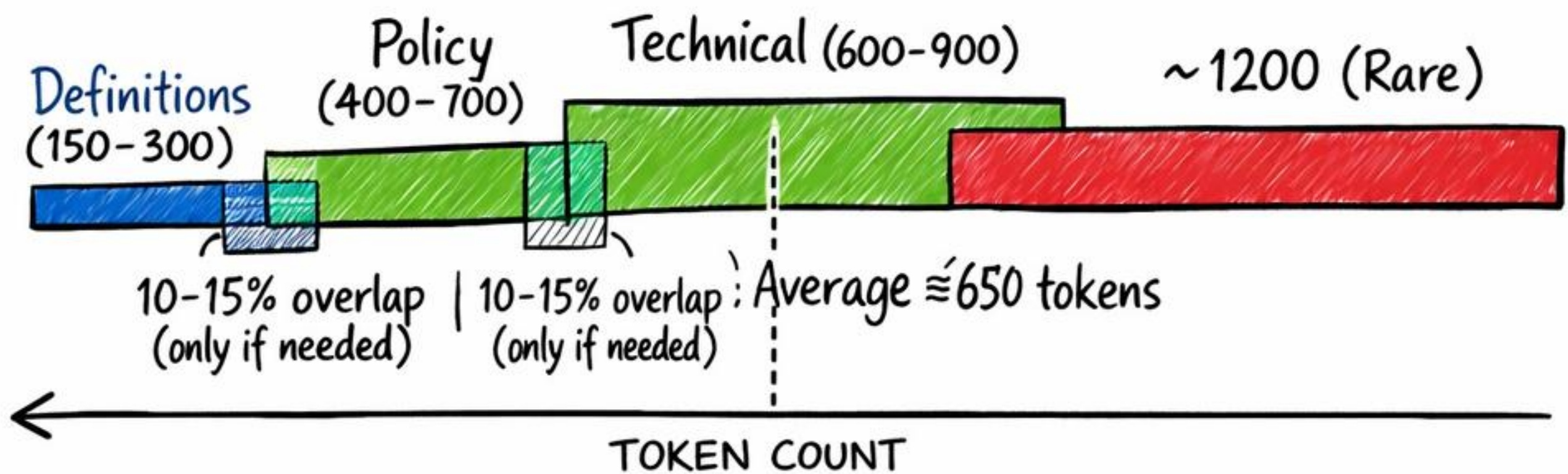
(MEANING-BASED SPLITTING)



Semantic chunking divides text based on meaning, not arbitrary length. Each chunk aligns with natural structure—headings, paragraphs, or topic boundaries—so it represents one coherent concept.

This allows each unit to stand independently during retrieval, improving clarity and contextual accuracy.

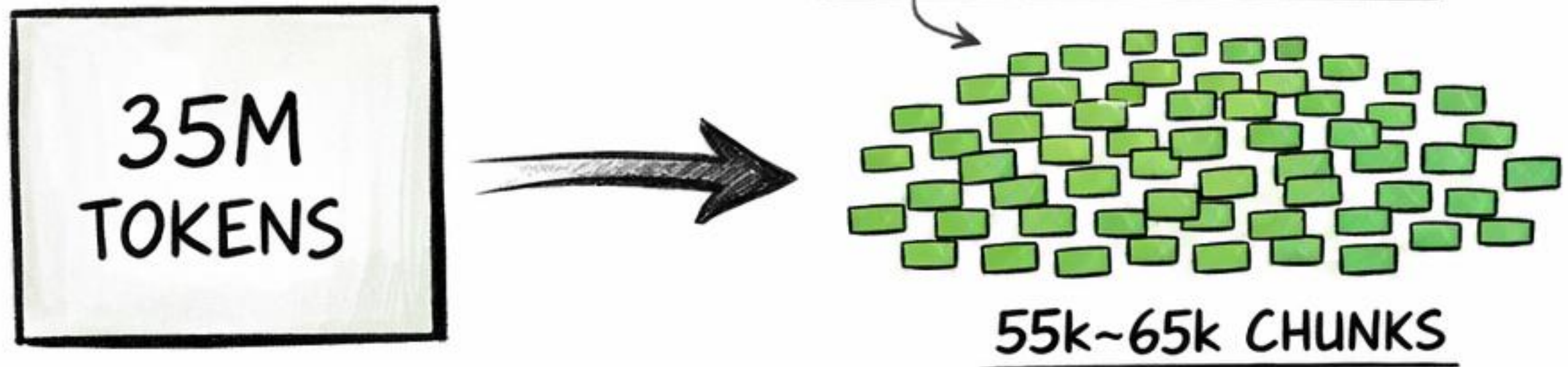
CHUNK SIZE DISTRIBUTION (NOT FIXED LENGTH)



Chunk sizes follow a distribution rather than a fixed limit.

Short definitions are smaller, technical explanations larger, and rare long sections extend further. The average effective size is about 650 tokens. Limited semantic overlap (10-15%) is applied only when continuity requires it.

SCALING BEHAVIOR OF SEMANTIC CHUNKING

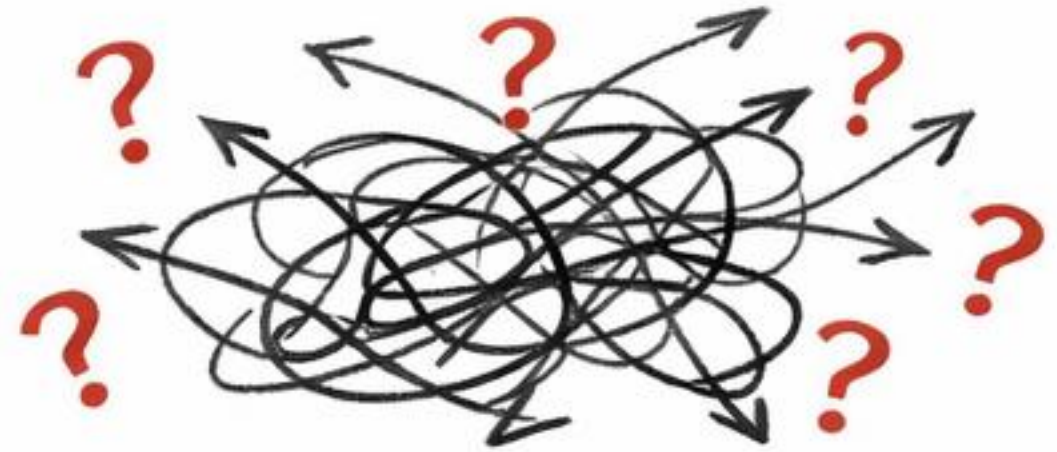


Quality > Quantity

HIGH QUALITY CHUNKS
→ CLEAN RETRIEVAL

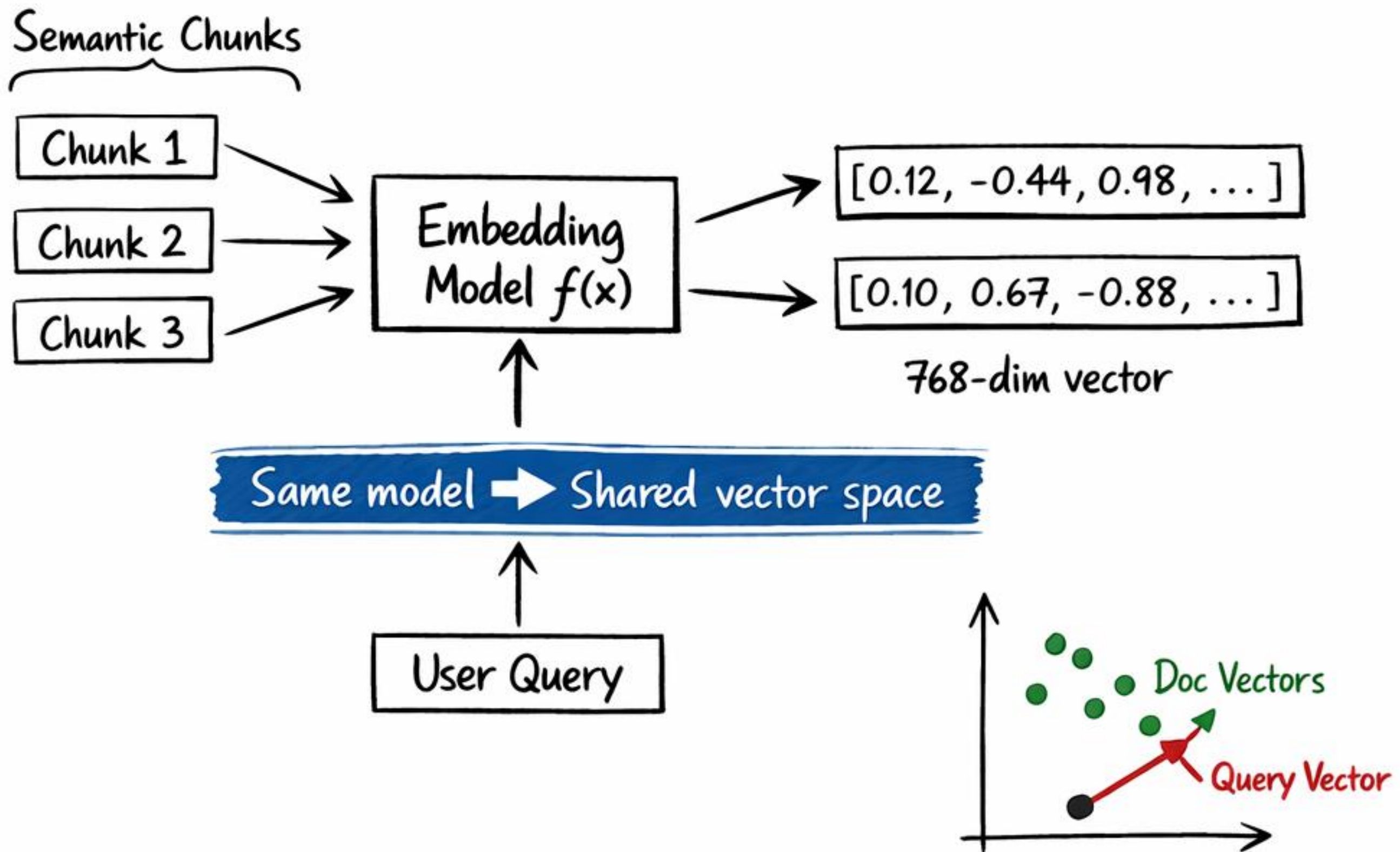


POOR CHUNKING → NOISE



Chunk count scales linearly with corpus size. From 35 million tokens, roughly 55,000–65,000 chunks are produced. However, retrieval performance depends primarily on chunk quality. Poor semantic boundaries introduce noise, regardless of scale, making careful chunk design more important than raw quantity.

EMBEDDING GENERATION (SEMANTIC ENCODING)



Each text chunk is transformed into a dense numerical vector capturing semantic meaning. The same embedding model is used for both stored documents and incoming queries. Because they share the same vector space, similarity can be computed using distance metrics like cosine similarity.

VECTOR DIMENSION & STORAGE COST



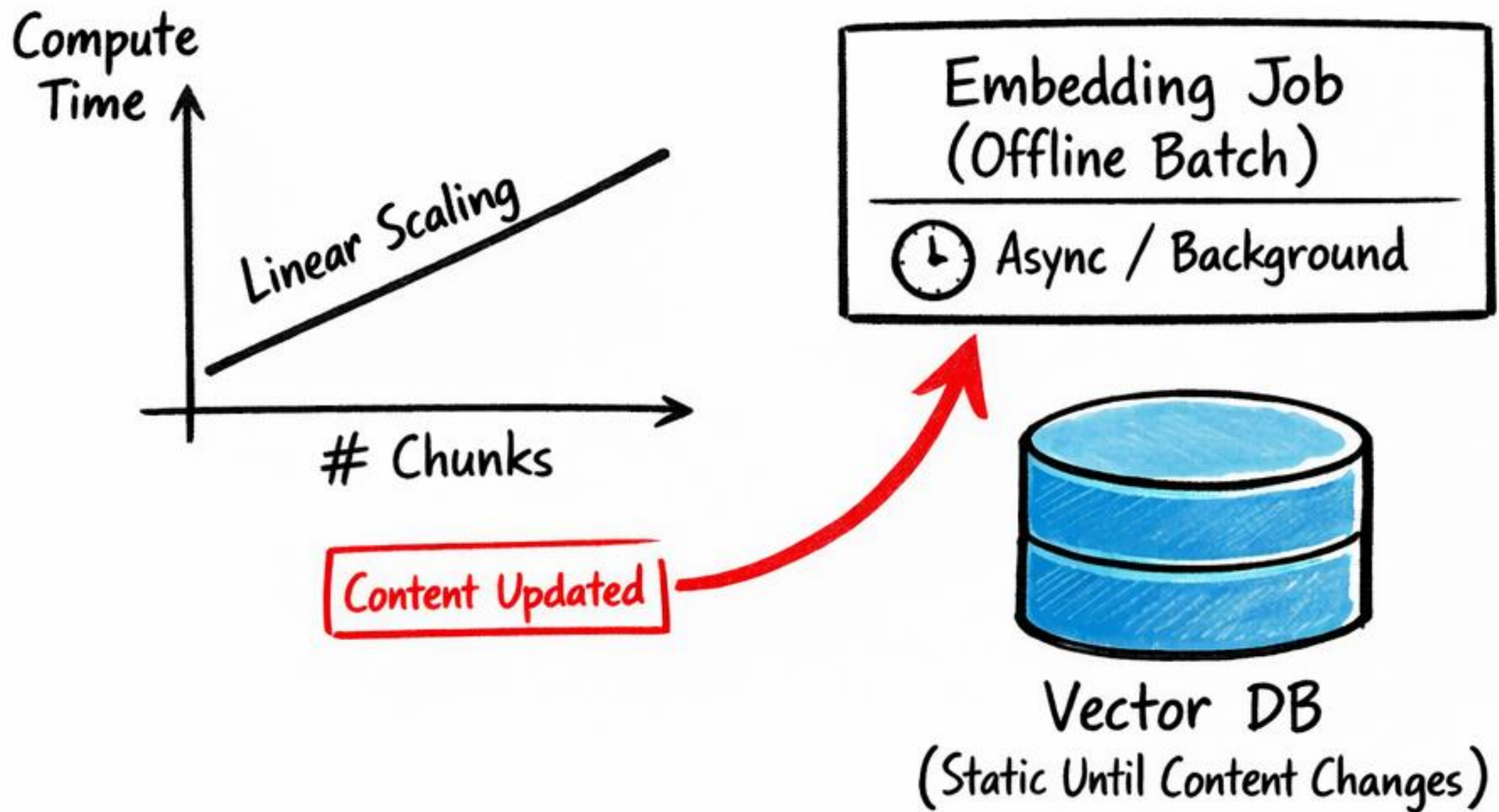
768 dims $\approx$ **3 KB** per vector

$$\begin{array}{r} 60,000 \text{ chunks} \\ \times 3 \text{ KB} \\ \hline = 180 \text{ MB} \end{array}$$

~180 MB raw vector data

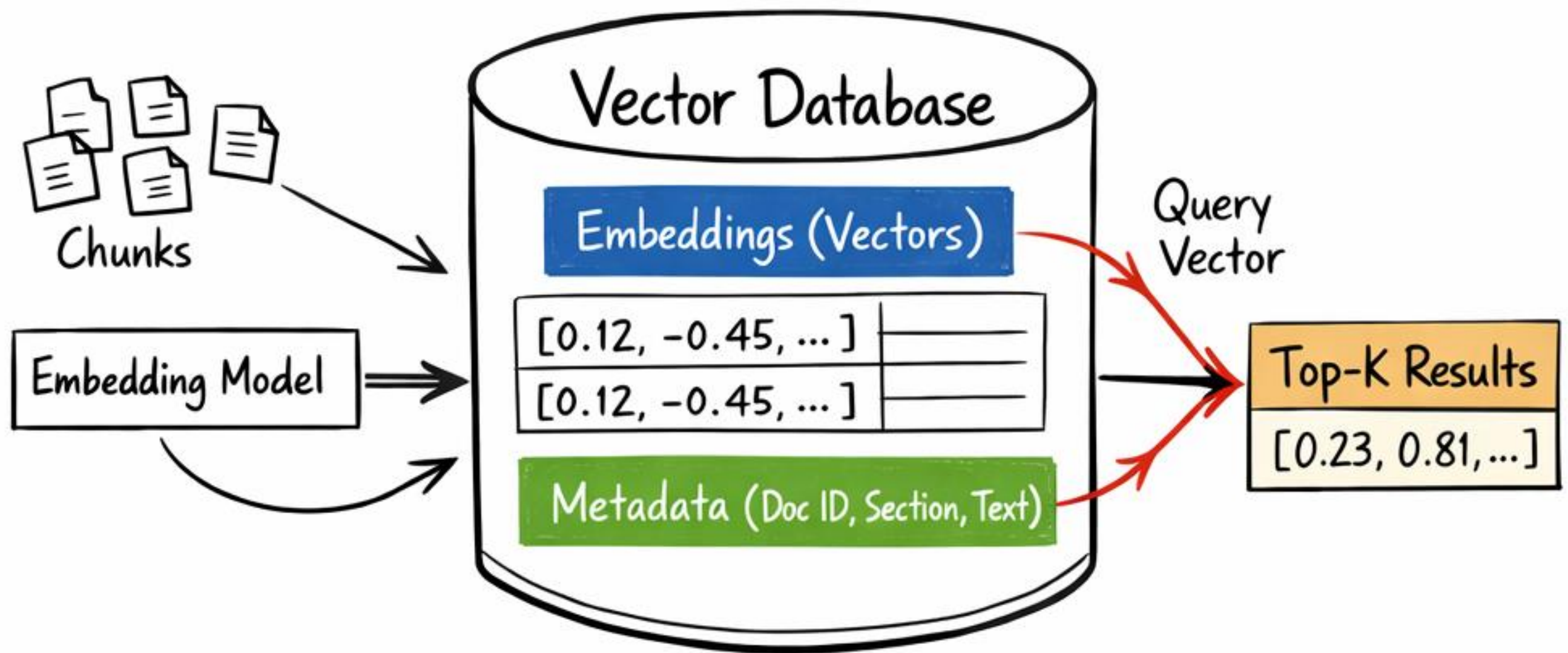
Each semantic chunk is stored as a 768-dimensional floating-point vector. At roughly **3 KB** per vector, storage grows linearly with the number of chunks. For 60,000 chunks, the total raw embedding storage is approximately 180 MB before indexing overhead.

SCALING BEHAVIOR OF EMBEDDING GENERATION



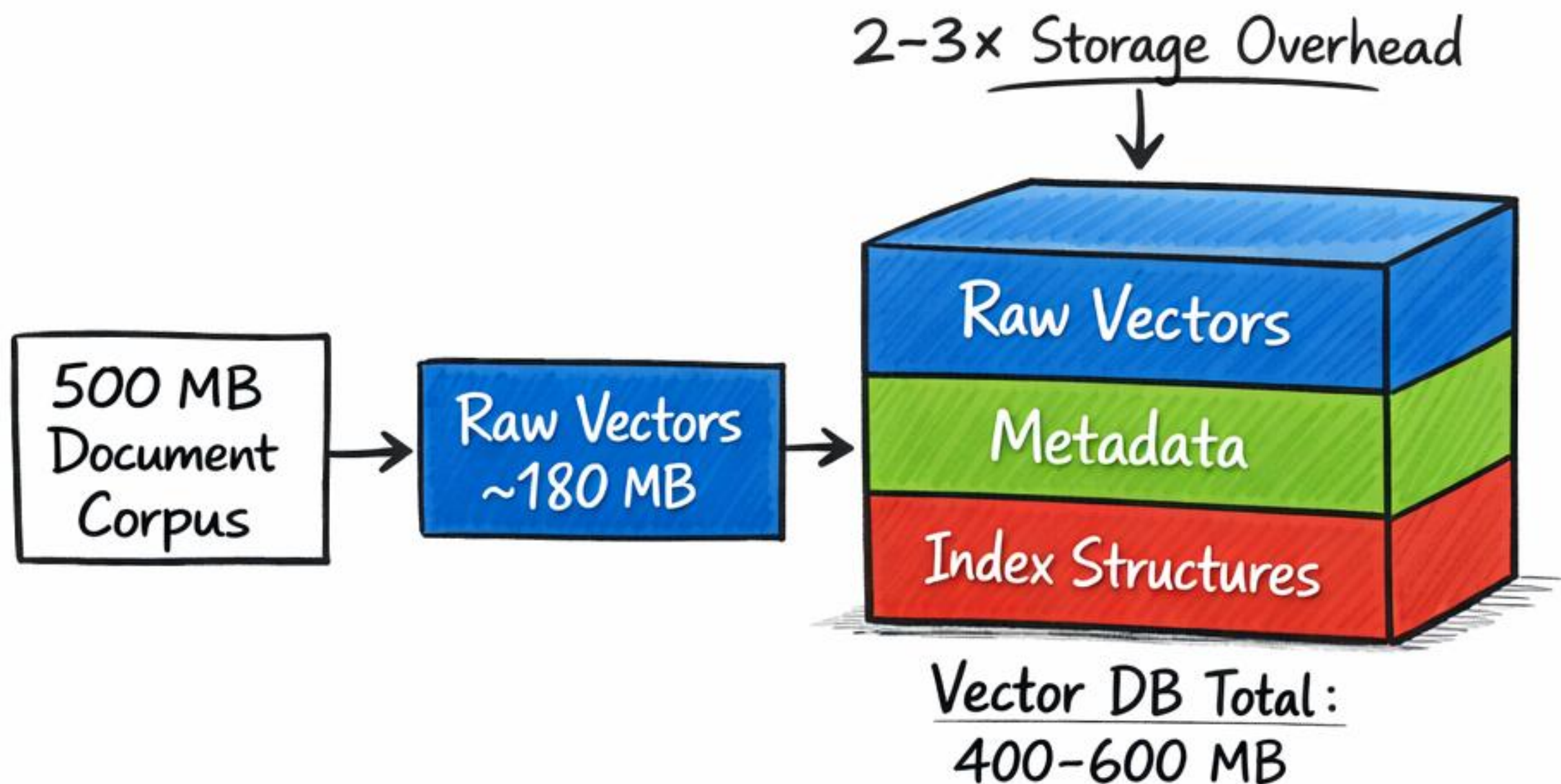
Embedding generation cost increases linearly with the number of chunks. Because this step is compute-intensive, it is typically performed offline or asynchronously. Once vectors are created and stored, they remain static and only need regeneration when the underlying content changes.

VECTOR DATABASE: ROLE IN RETRIEVAL



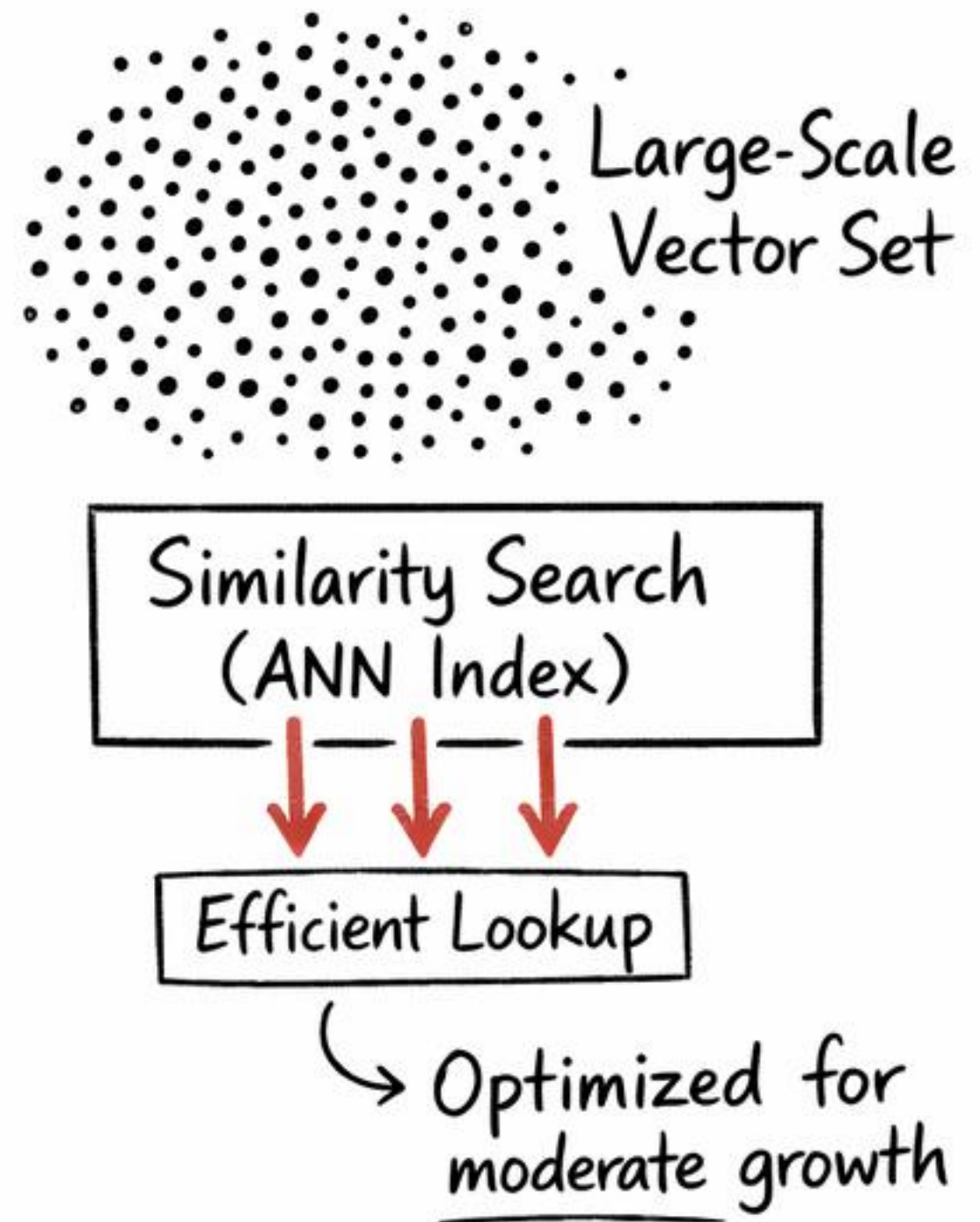
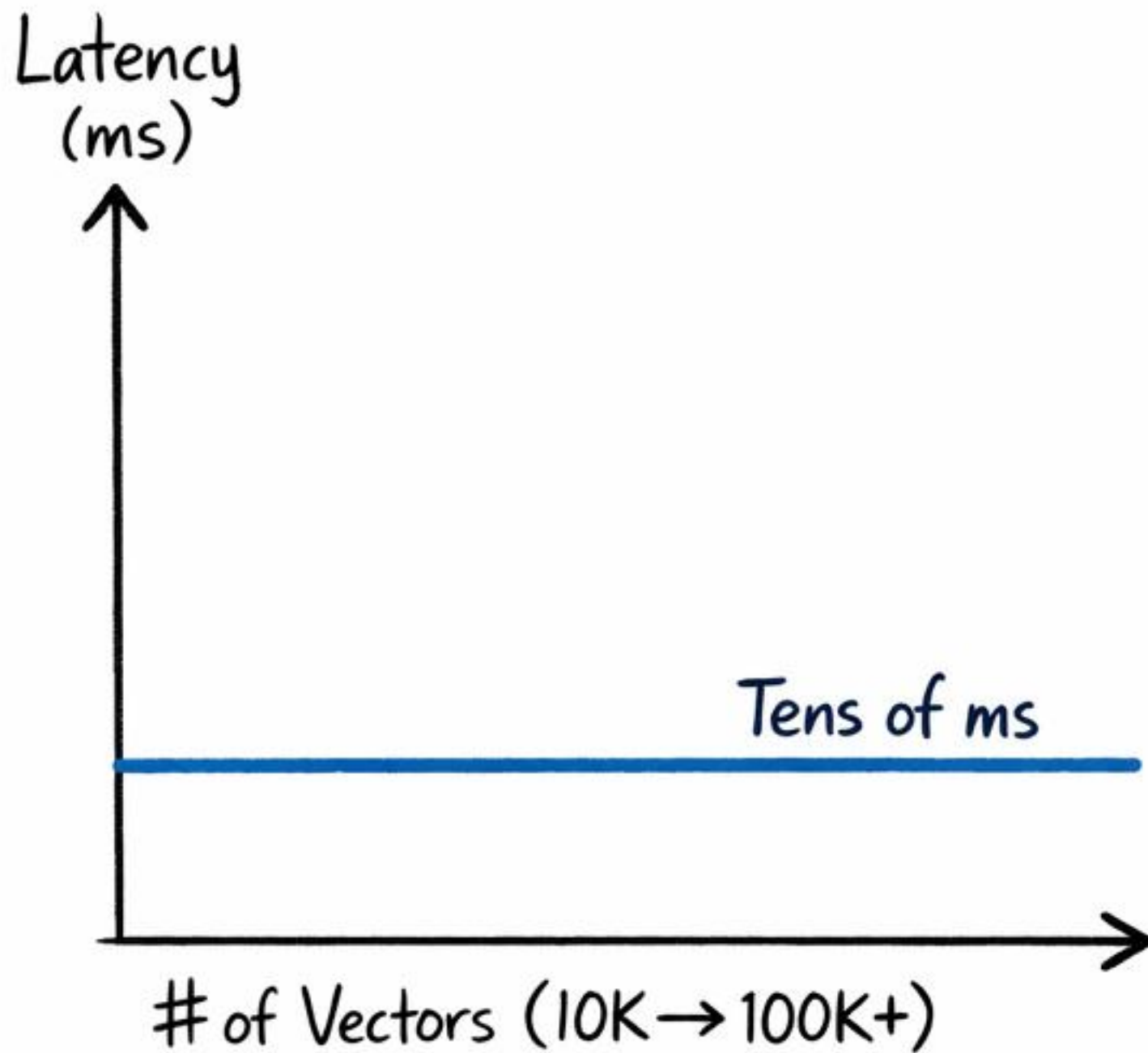
The vector database stores high-dimensional embeddings along with metadata such as document IDs and text chunks. When a query embedding arrives, it performs fast similarity search to retrieve the most relevant vectors. It acts as the memory layer enabling semantic retrieval in RAG systems.

STORAGE FOOTPRINT & OVERHEAD



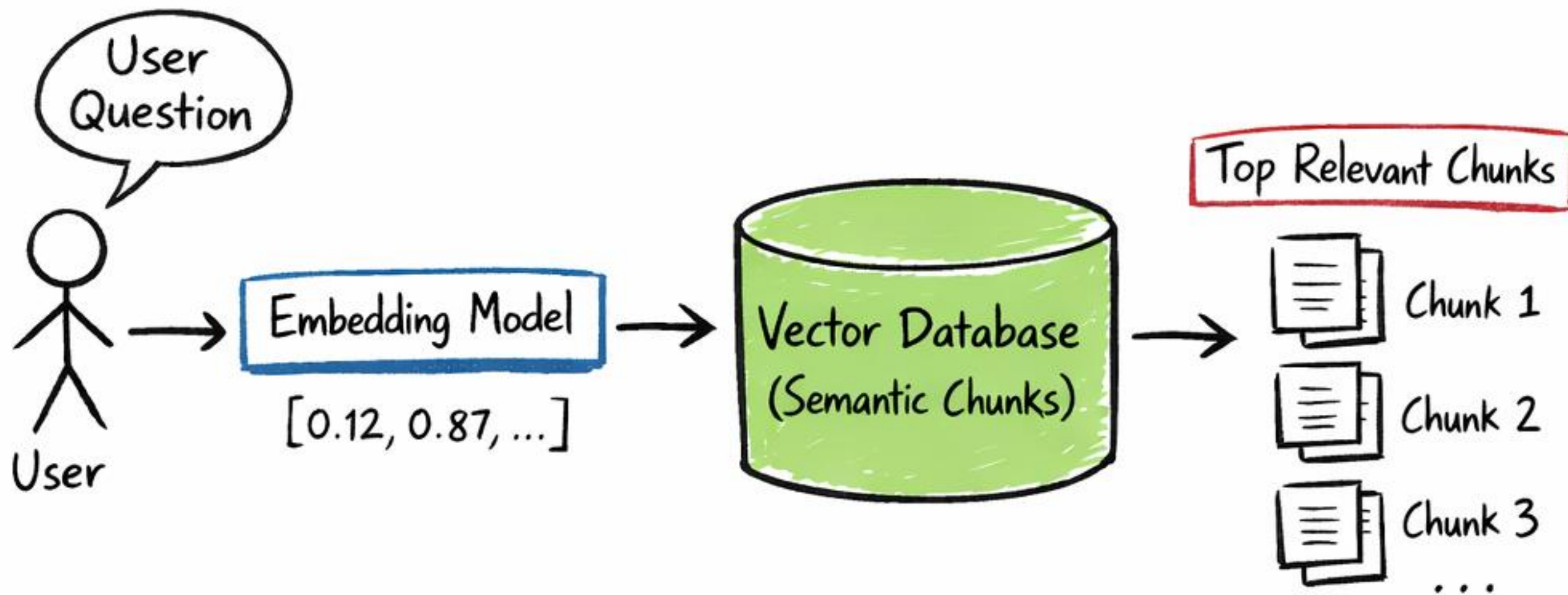
Beyond storing raw embedding vectors, the database maintains indexing structures for fast search and metadata for retrieval context. These additional components introduce a 2-3x storage overhead. Thus, a 180 MB embedding set may require 400-600 MB in the full vector database system.

SCALING & QUERY LATENCY



Vector databases use optimized indexing methods for approximate nearest neighbor search. This allows tens to hundreds of thousands of vectors to be queried efficiently. At this scale, latency remains in the tens of milliseconds and does not increase significantly with moderate dataset growth.

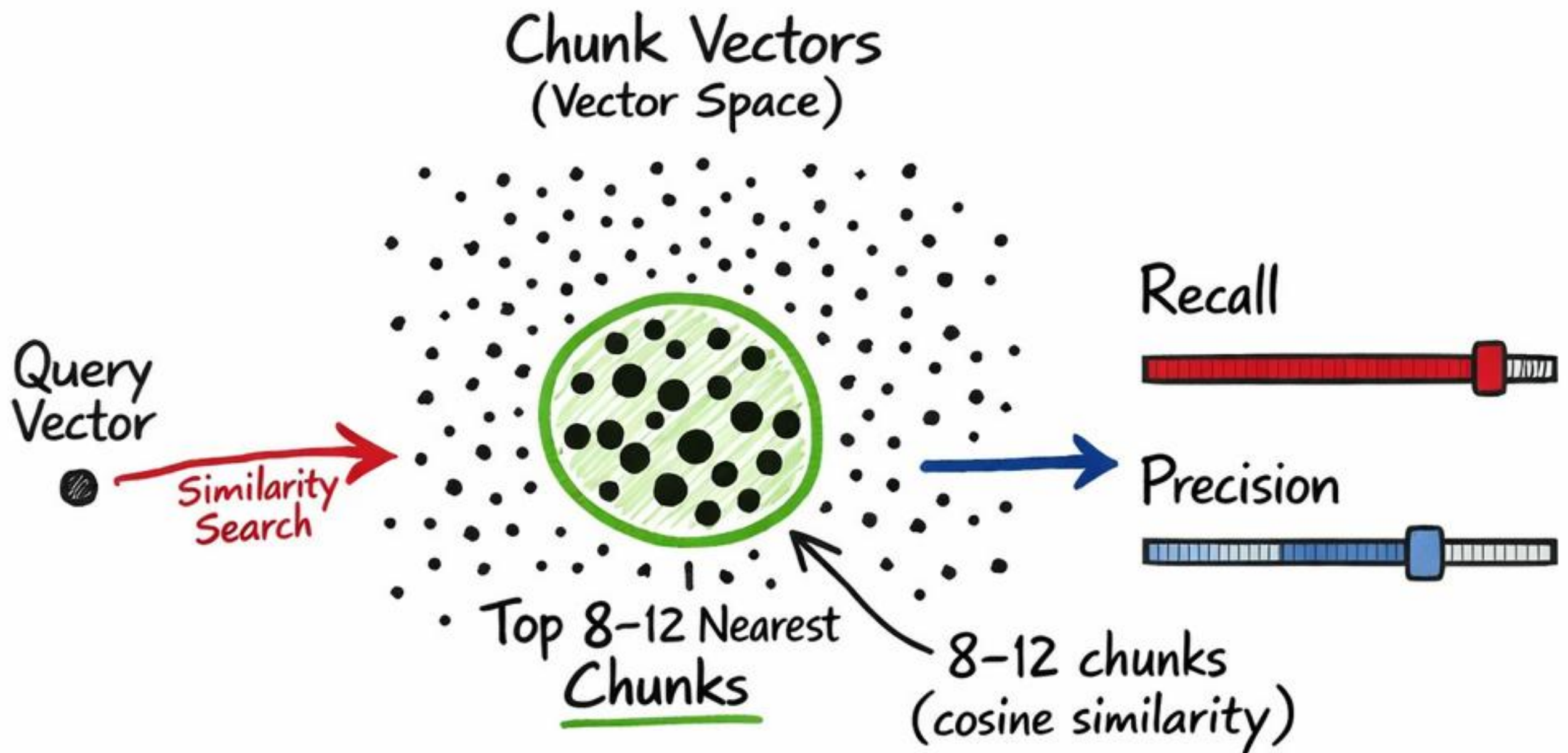
QUERY PROCESSING PIPELINE



User questions are converted into vector embeddings that capture semantic meaning. The system compares this query vector against stored document chunk vectors in a database. Using similarity search, it retrieves the most semantically relevant chunks to support accurate answer generation.

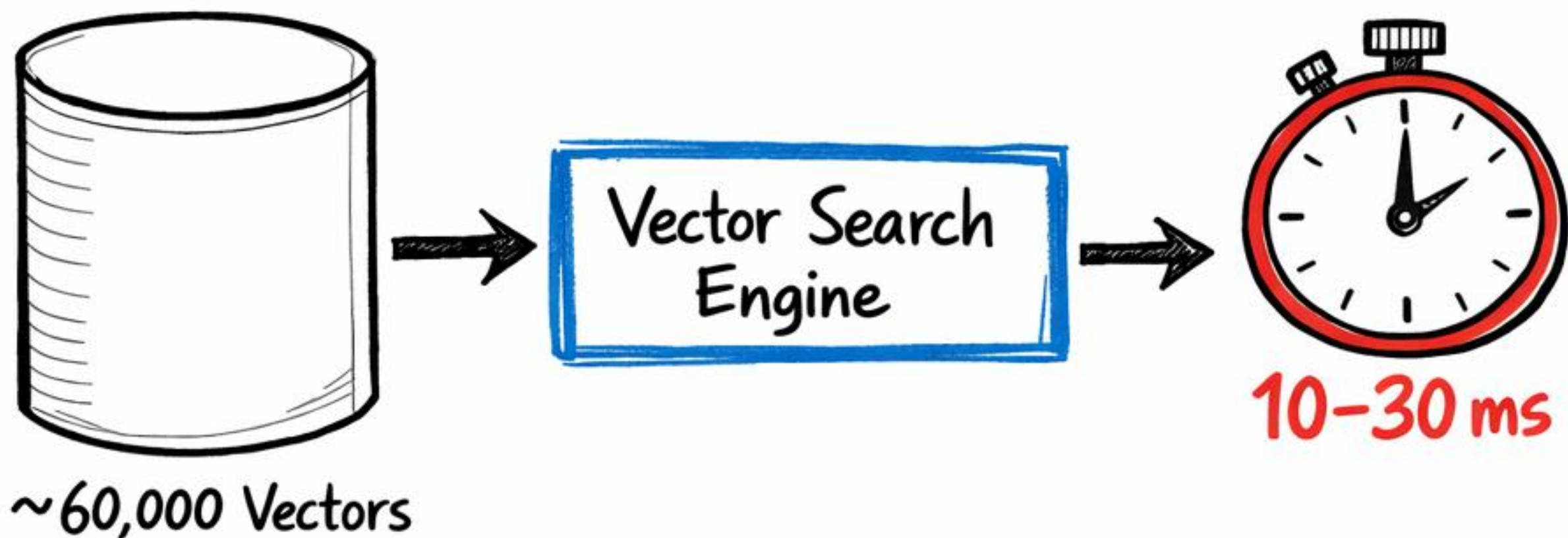
RETRIEVAL STRATEGY:

BALANCING RECALL & PRECISION

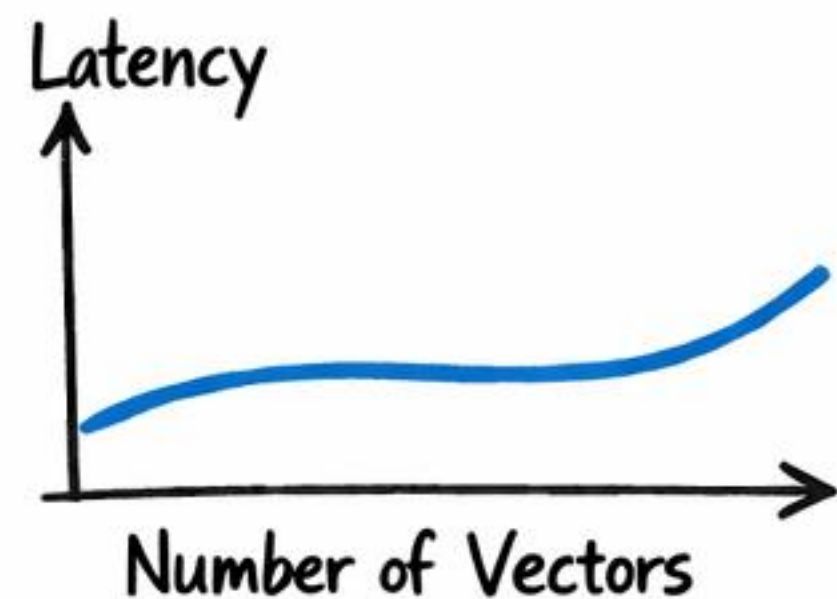


To balance recall and precision, the system retrieves 8-12 semantically nearest chunks using **cosine similarity**. A slightly higher initial recall helps compensate for variations in chunk size and granularity, ensuring important context is not missed before downstream filtering or generation.

SCALING CHARACTERISTICS OF VECTOR SEARCH



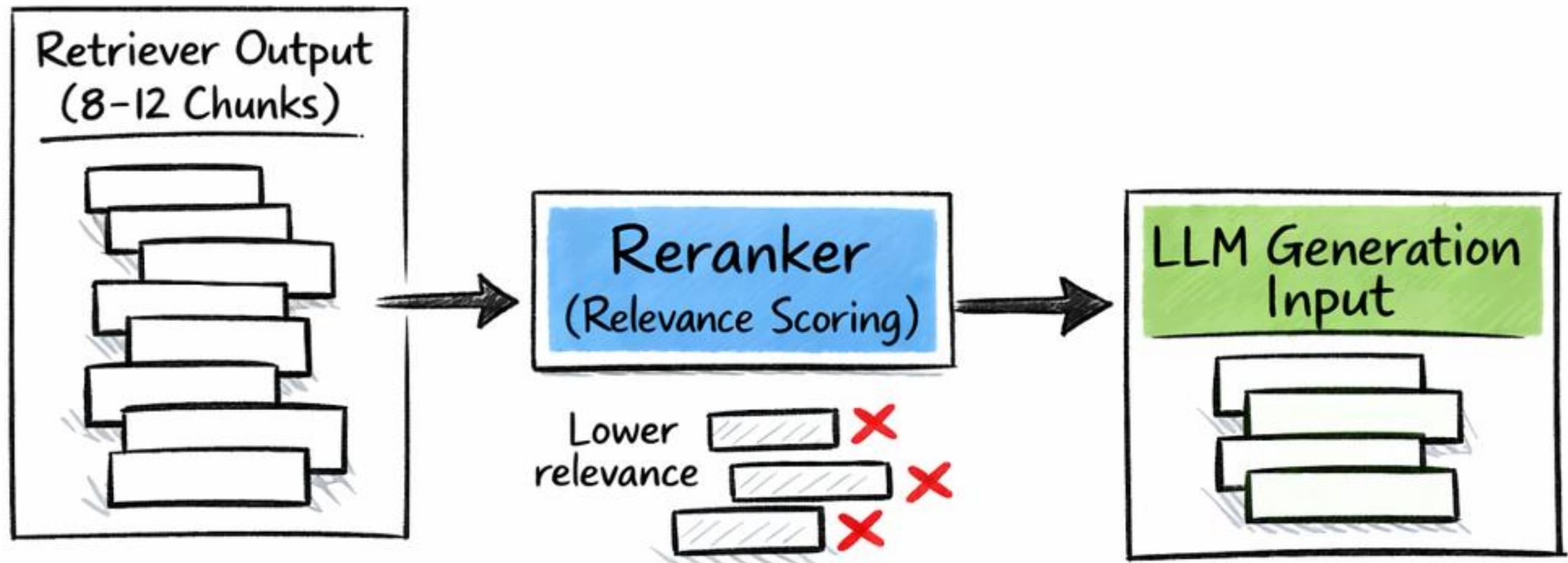
- Query Embedding = Low Latency
- Similarity Search = Efficient



Both query embedding and similarity search are computationally efficient. Even with tens of thousands of stored vectors, modern vector databases can retrieve nearest neighbors in 10-30 milliseconds. This low latency enables real-time semantic retrieval at production scale.

RERANKING:

SELECTING THE MOST RELEVANT CONTEXT



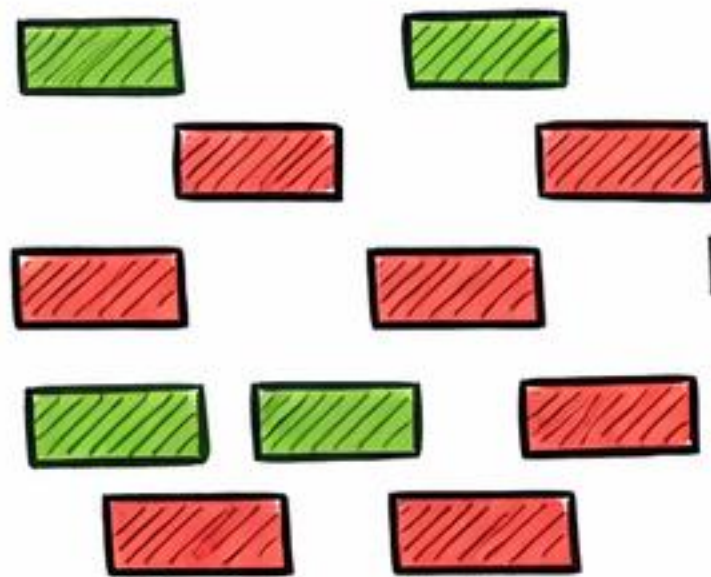
Initial retrieval returns multiple candidate chunks, but not all are equally relevant.

The reranker scores and filters these candidates, selecting only the most useful context for the language model.

This focused selection significantly improves answer quality by ensuring generation is grounded in the strongest supporting evidence.

TWO-STAGE CONTEXT SELECTION

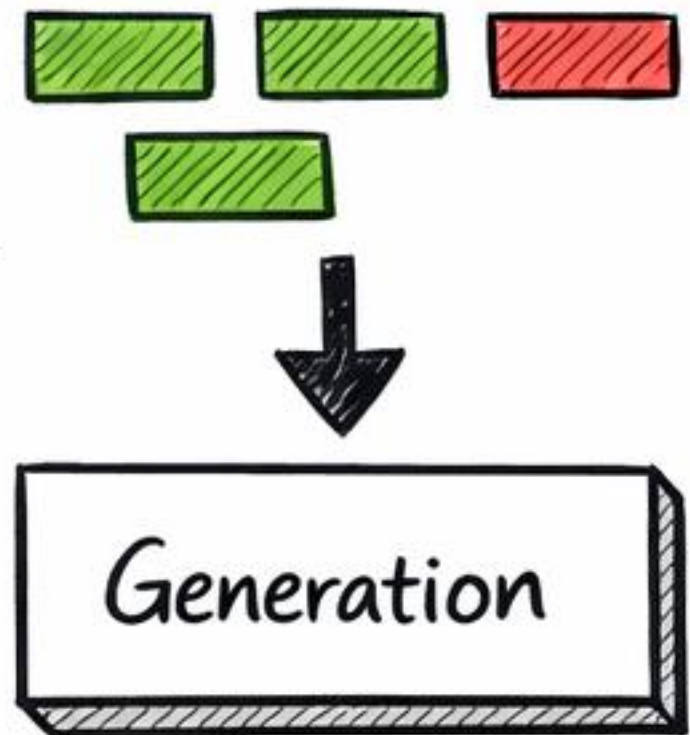
Stage 1 - Retrieval
(8-12 chunks)



Stage 2 - Reranking
(Scoring & Sorting)

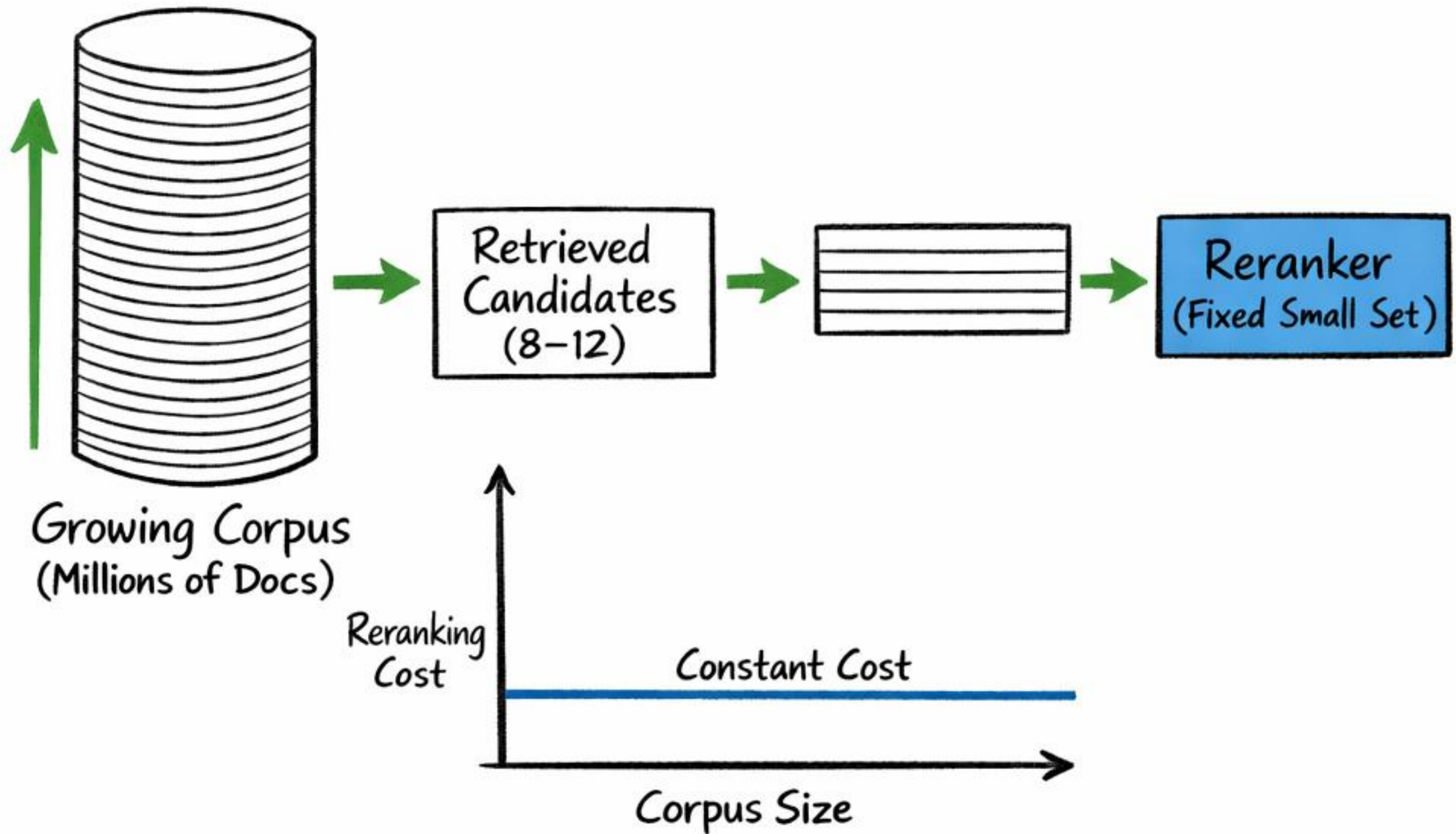


Final Context
(4-6 chunks)



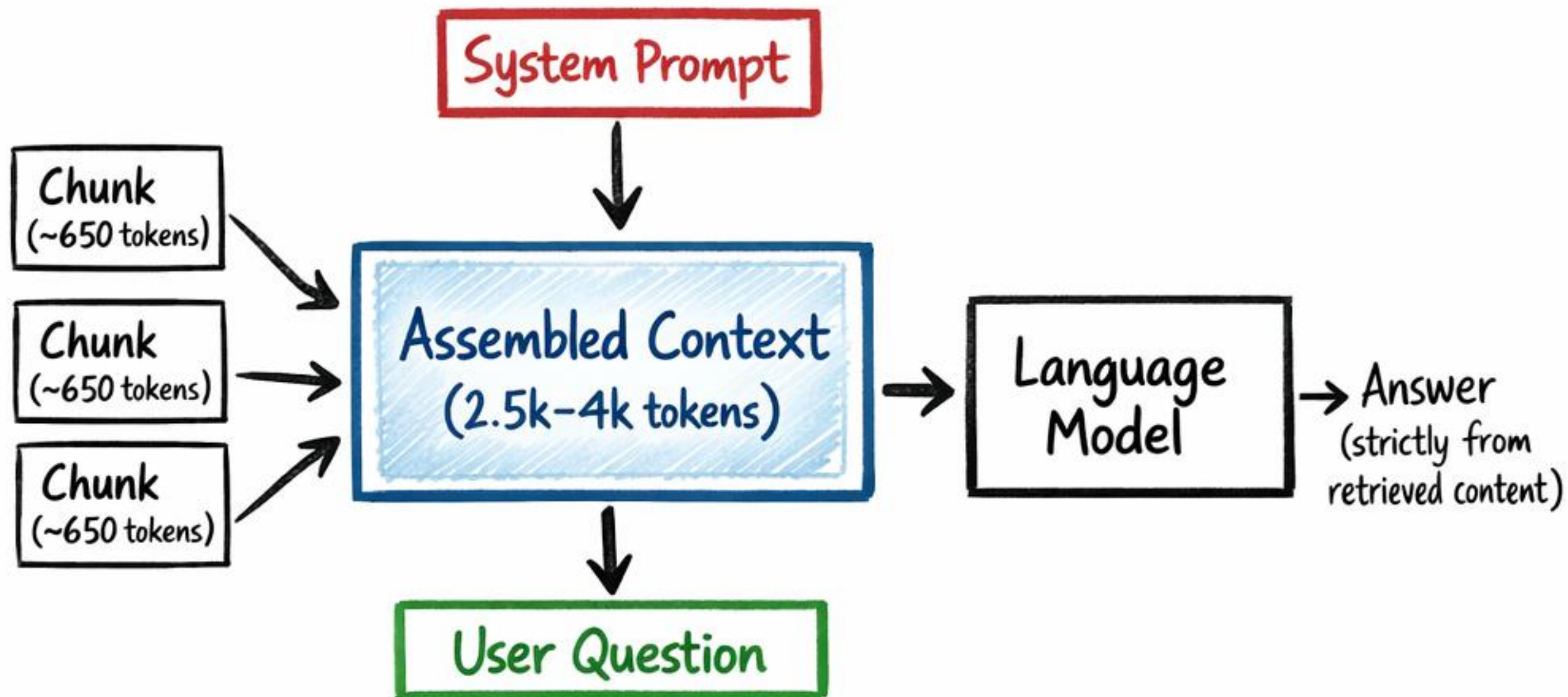
The system first retrieves a broad set of candidate chunks to maximize recall. The reranker then evaluates and orders them by relevance. Only the top-ranked 4-6 chunks are passed to the model. This two-stage process balances coverage and precision, improving answer accuracy without increasing storage size.

RERANKING SCALES WITH QUERY, NOT CORPUS



Reranking operates only on a small set of retrieved candidates, not the entire corpus. Even as the dataset grows to millions of documents, the reranker evaluates just 8-12 chunks per query. Its computational cost depends on query complexity, keeping performance stable and bounded at scale.

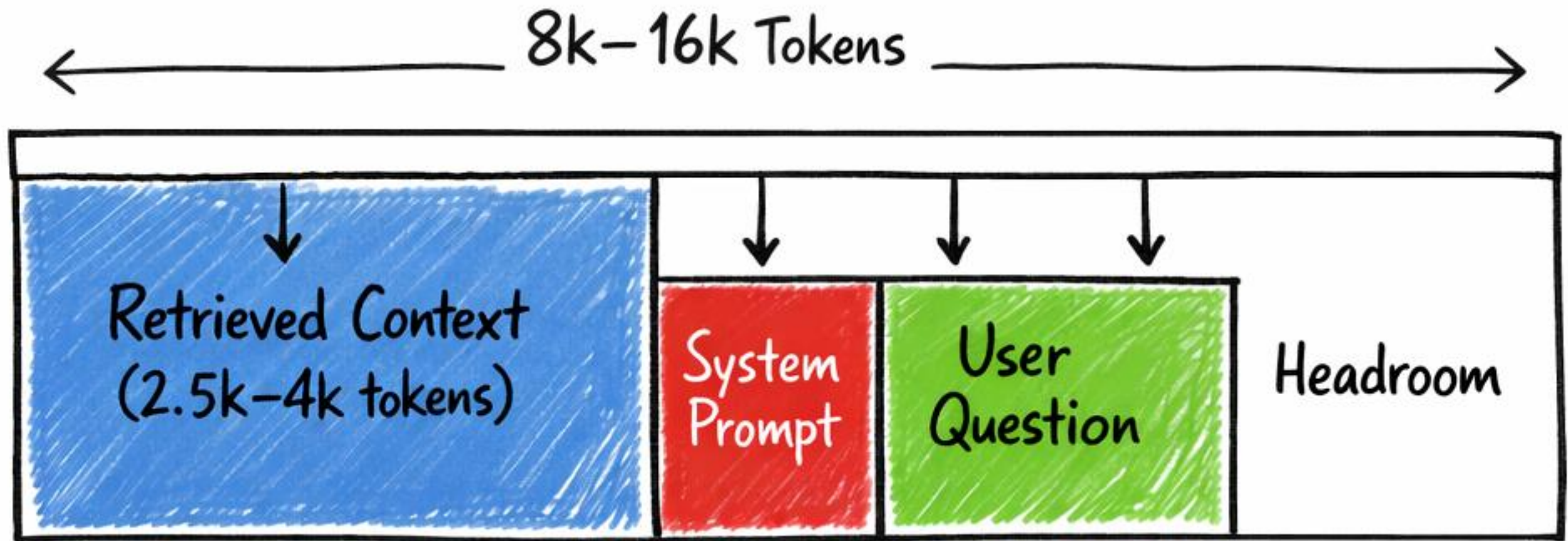
CONTEXT ASSEMBLY → GROUNDED GENERATION



Retrieved chunks are merged into a single context window along with system instructions and the user's question.

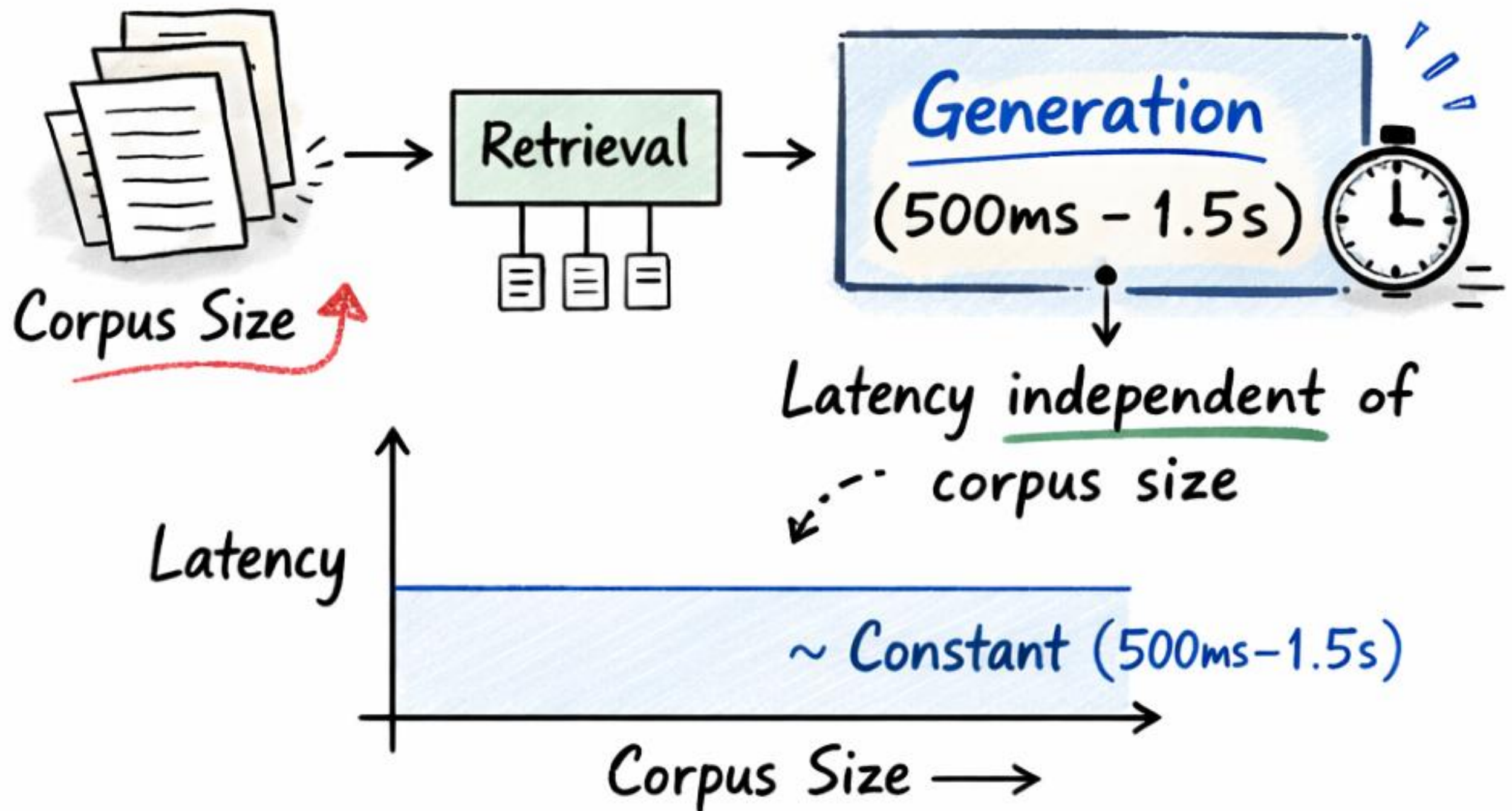
The language model is constrained to generate answers strictly from this assembled evidence. This grounding step ensures factual consistency and prevents the model from relying on outside or parametric knowledge.

TOKEN BUDGET WITHIN 8K-16K WINDOW



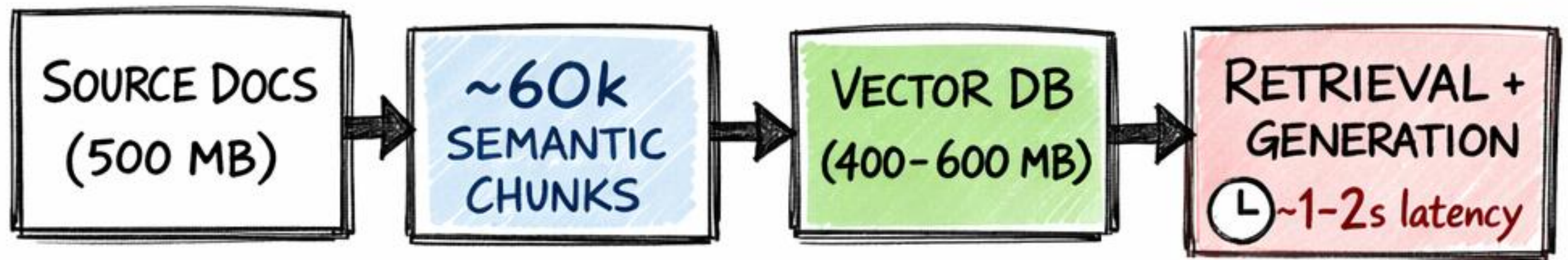
The assembled context must fit within the model's maximum token window. Retrieved chunks typically consume 2.5k-4k tokens, leaving space for system instructions and user input. Careful budgeting ensures the model operates within limits while maximizing useful retrieved evidence.

GENERATION LATENCY & SCALING



Generation is the dominant latency source, typically 500ms-1.5s. Unlike retrieval, generation time does not increase with corpus size. Once context is assembled, response time depends on model architecture & deployment — not total data volume.

END-TO-END SCALING @ 500MB



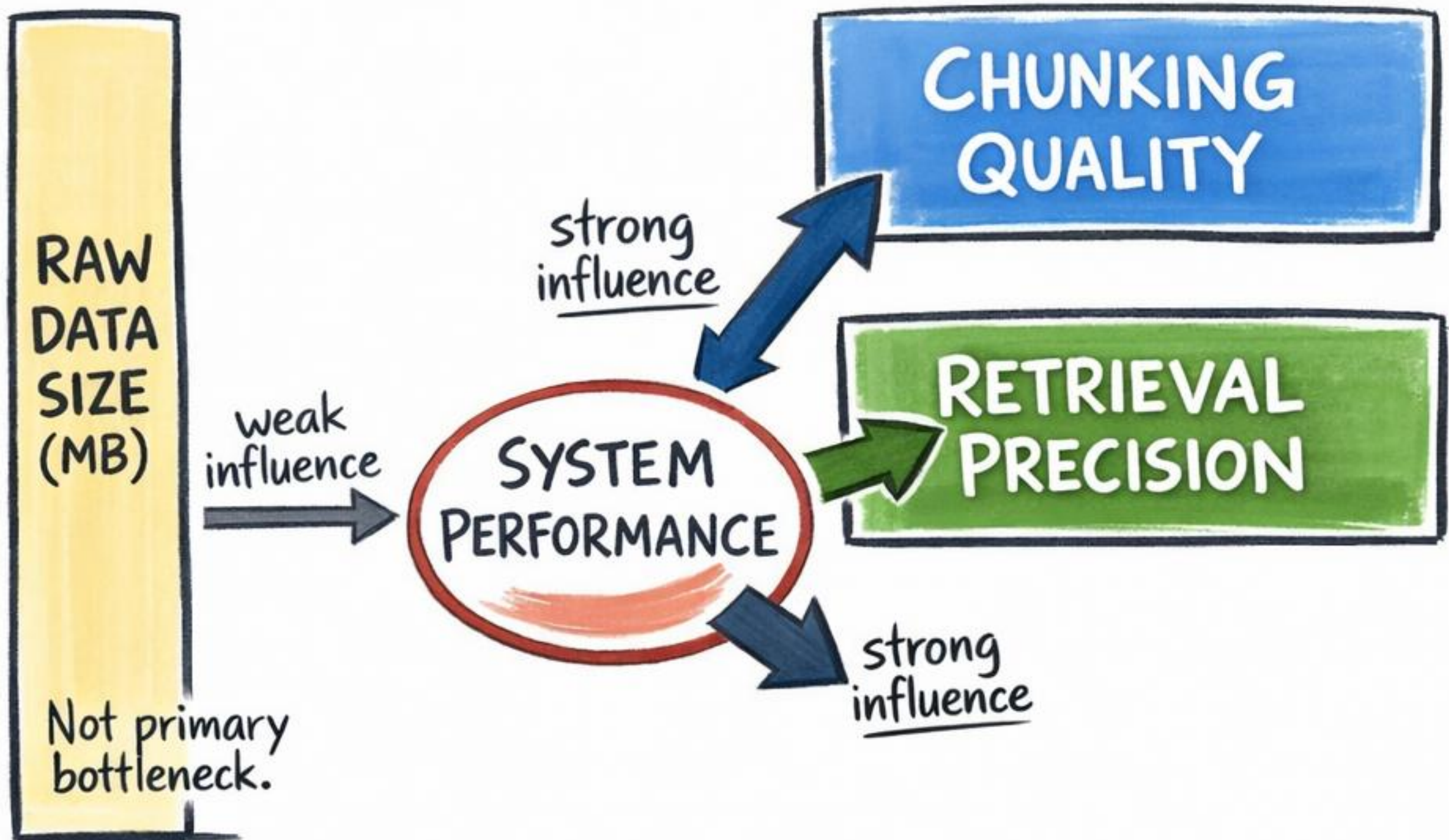
- Retrieval cost $\propto$ #chunks (linear)
- Still small vs total response time

Even at 500 MB of documents, the system remains efficient.

Documents are converted into ~60k semantic chunks, stored in 400-600 MB vector database. Retrieval scales linearly with chunk count, but total latency remains around 1-2s, keeping production performance stable.

WHAT LIMITS SCALABILITY?

in osheenkhare



System scalability depends more on how well documents are chunked and how precisely relevant chunks are retrieved than on total data size. Good chunking reduces noise, and accurate retrieval limits unnecessary computation. Raw storage size matters far less than information organization and access efficiency.

Keeping Up with AI?
Follow Me

Hand-drawn doodles including a grid, triangles, and swirls surrounding the text:
@osheenkhare

